

Data Preparation

Zhirong Yang

Prepare tabular data

Tabular data: example

ID	Color	Quality	Order Date	Location	Units	Amount
a1011	red	good	11/06/2016	BJ	11	563.19
a1013	blue	best	03/01/2017	GZ	23	1286.21
a1017	white	ok	13/03/2016	DG	12	345.62
a1022	red	bad	22/01/2016	SA	8	190.95
a1025	red	best	31/01/2017	BJ	98	3290.87
a1029	green	good	10/02/2017	GZ	30	1090.87
...	...	...	...	...	...	...
a3362	white	ok	28/02/2017	FS	20	600.28

Tabular data: example

ID	Color	Quality	Order Date	Location	Units	Amount
a1011	red	good	11/06/2016	BJ	11	563.19
a1013	blue	best	03/01/2017	GZ	23	1286.21
a1017	white	ok	13/03/2016	DG	12	345.62
a1022	red	bad	22/01/2016	SA	8	190.95
a1025	red	best	31/01/2017	BJ	98	3290.87
a1029	green	good	10/02/2017	GZ	30	1090.87
...	...	...	...	...	...	...
a3362	white	ok	28/02/2017	FS	20	600.28

Why tabular data?

- ▶ Excel is widely used
- ▶ Raw data is often stored in SQL tables
- ▶ Easy to add/remove fields

Tabular data: example

ID	Color	Quality	Order Date	Location	Units	Amount
a1011	red	good	11/06/2016	BJ	11	563.19
a1013	blue	best	03/01/2017	GZ	23	1286.21
a1017	white	ok	13/03/2016	DG	12	345.62
a1022	red	bad	22/01/2016	SA	8	190.95
a1025	red	best	31/01/2017	BJ	98	3290.87
a1029	green	good	10/02/2017	GZ	30	1090.87
...	...	...	...	...	...	...
a3362	white	ok	28/02/2017	FS	20	600.28

Why tabular data?

- ▶ Excel is widely used
- ▶ Raw data is often stored in SQL tables
- ▶ Easy to add/remove fields
- ▶ Need to convert the tabular data to a numeric matrix

Scalar data types

Scalar = one-element or individual (result is invariant by changing scalar columns)

- ▶ nominal
 - ▶ Examples: ID numbers, eye color, zip codes
- ▶ ordinal (has order information)
 - ▶ Examples: rankings (e.g., taste of potato chips on a scale from 1-10), grades, height in tall, medium, short
- ▶ interval (ordinal + plus/minus operations)
 - ▶ Examples: calendar dates, temperatures in Celsius or Fahrenheit.
- ▶ ratio (interval + multiply/divide operations)
 - ▶ Examples: temperature in Kelvin, length, time, counts

One-Hot encoding of categorical data

- ▶ Ordinal, interval, and ratio data are usually treated as numeric
- ▶ For nominal
 - ▶ ID and other unique attributes do not contribute to ML
 - ▶ Reduce number of values
 - ▶ e.g. zip codes to areas
- ▶ One-hot encoding (e.g. Color)

ID	red	blue	white	yellow	green
a1011	1	0	0	0	0
a1013	0	1	0	0	0
a1017	0	0	1	0	0
a1022	0	0	0	1	0
a1025	0	0	0	1	0
a1029	0	0	0	0	1
...	...	...	...	...	...
a3362	0	0	1	0	0

- ▶ In software
 - ▶ `pandas.get_dummies`
 - ▶ `sklearn.preprocessing.OneHotEncoder`

One-Hot encoding of categorical data: demo

Structured data

Structured = result can alter by changing element order

- ▶ text
- ▶ speech
- ▶ other sequences
- ▶ image
- ▶ links

Bag-of-Words (BoW) representation

- ▶ Term Frequency (TF)
 - ▶ $TF(d, t)$ is the count of the t th term in the d th document
 - ▶ Remove stop-list and weird words

Bag-of-Words (BoW) representation

- ▶ Term Frequency (TF)
 - ▶ $TF(d, t)$ is the count of the t th term in the d th document
 - ▶ Remove stop-list and weird words
- ▶ Term frequency-Inverse document frequency (TFIDF)
 - ▶ Frequent terms dominate TF
 - ▶ TFIDF weights TF by IDF:

$$TFIDF(d, t) = TF(d, t) \cdot \log\left(1 + \frac{N}{n_t}\right)$$

N : total number of documents

n_t : number of documents where the term t appears

Bag-of-Words (BoW) representation

- ▶ Term Frequency (TF)
 - ▶ $TF(d, t)$ is the count of the t th term in the d th document
 - ▶ Remove stop-list and weird words
- ▶ Term frequency-Inverse document frequency (TFIDF)
 - ▶ Frequent terms dominate TF
 - ▶ TFIDF weights TF by IDF:

$$TFIDF(d, t) = TF(d, t) \cdot \log\left(1 + \frac{N}{n_t}\right)$$

N : total number of documents

n_t : number of documents where the term t appears

- ▶ See NLTK or TfidfVectorizer in sklearn

Bag-of-Words (BoW) representation

- ▶ Term Frequency (TF)
 - ▶ $TF(d, t)$ is the count of the t th term in the d th document
 - ▶ Remove stop-list and weird words
- ▶ Term frequency-Inverse document frequency (TFIDF)
 - ▶ Frequent terms dominate TF
 - ▶ TFIDF weights TF by IDF:

$$TFIDF(d, t) = TF(d, t) \cdot \log\left(1 + \frac{N}{n_t}\right)$$

N : total number of documents

n_t : number of documents where the term t appears

- ▶ See NLTK or TfidfVectorizer in sklearn
- ▶ Pros and cons
 - + easy to compute at scale
 - + reasonably good result with TFIDF
 - term order is not encoded
 - need dimensionality reduction
- ▶ Applicable for other count data and sequence of symbols

Mel-frequency cepstrum (MFCC) for acoustic data

- ▶ MFCC represents the short-term power spectrum of a sound
 - ▶ It uses a linear cosine transform of a log power spectrum on a nonlinear mel scale of frequency
 - ▶ Tutorial available at <http://practicalcryptography.com/miscellaneous/machine-learning/guide-mel-frequency-cepstral-coefficients-mfcc/>
- ▶ MFCC is the most widely used method for speech
- ▶ In python

```
1 from python_speech_features import mfcc
2 freq_samp, audio = wavfile.read("sample.wav")
3 features_mfcc = mfcc(audio, freq_samp)
```

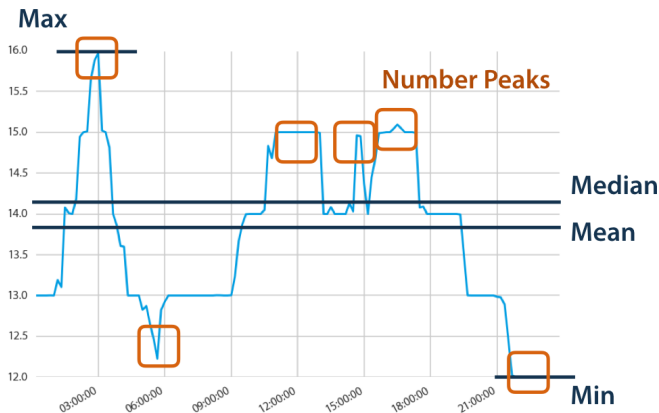
 - ▶ Outputs a matrix sized $\#frames \times \#cepstrum$ (default 13)
 - ▶ Adjusts parameters to change $\#frames$ (e.g. winlen, samplerate)
- ▶ Other signal processing methods (e.g. logf and scattering)

TSFRESH for time series data

- ▶ Can we directly use curves (if lengths are the same)?

TSFRESH for time series data

- ▶ Can we directly use curves (if lengths are the same)?
 - ▶ No, it is sensitive to small variation
- ▶ TSFRESH automatically extracts 100s of features from time series, e.g.



TSFRESH demo

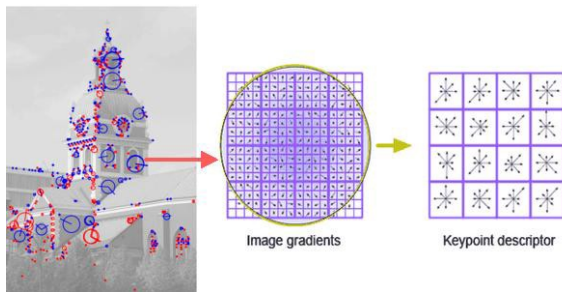
(Additional Reading) TS2Vec

- ▶ A more recent feature extraction method for time series
- ▶ It is becoming the state of the art
- ▶ Paper: [TS2Vec: Towards Universal Representation of Time Series](#)
- ▶ [Official GitHub Repository](#)
- ▶ [A more handy wrapper](#)

Feature extraction for images

- ▶ Computer vision is far beyond this course; We only touch a tip of the iceberg
 - ▶ check TDT4265 for more details
- ▶ OpenCV package (`cv2`) provides rich functionality
- ▶ Low-level feature extraction
 - ▶ fast; mainly based on signal processing
 - ▶ e.g. color, texture, shape, edges/corners
 - ▶ `skimage.feature` provides handy wrappers
 - ▶ See e.g.
 - ▶ `cv2.calcHist`
 - ▶ `skimage.feature.local_binary_pattern`
 - ▶ `skimage.feature.shape_index`

Scale Invariant Feature Transform (SIFT)



- ▶ A classical and widely used method
- ▶ Each image is described by a set of keypoint descriptors
- ▶ Can be treated as “visual words” after discretization
- ▶ See also
 - ▶ SURF and FAST (for faster extraction)
 - ▶ BRIEF (for shorter representation)
 - ▶ ORB (for free use)

(Additional reading) Pretrained models

- ▶ Use pretrained deep neural networks
 - ▶ Conventionally, apply supervised training for individual structured fields, then use the network output as representation. This is actually one of the stacking approaches.
 - ▶ Recently, use self-supervised pretraining
 - ▶ BERT or GPT for text
 - ▶ ViT/DINO/iBOT for images
 - ▶ [HuggingFace](#) has many pretrained models for X2vec

(Additional reading) Pretrained models

- ▶ Use pretrained deep neural networks
 - ▶ Conventionally, apply supervised training for individual structured fields, then use the network output as representation. This is actually one of the stacking approaches.
 - ▶ Recently, use self-supervised pretraining
 - ▶ BERT or GPT for text
 - ▶ ViT/DINO/iBOT for images
 - ▶ [HuggingFace](#) has many pretrained models for X2vec
- ▶ Pros and cons
 - + can extract comprehensive information
 - + from end to end
 - expensive in training time, tuning effort and computing resource
 - hard to explain so far

(Additional reading) Pretrained models

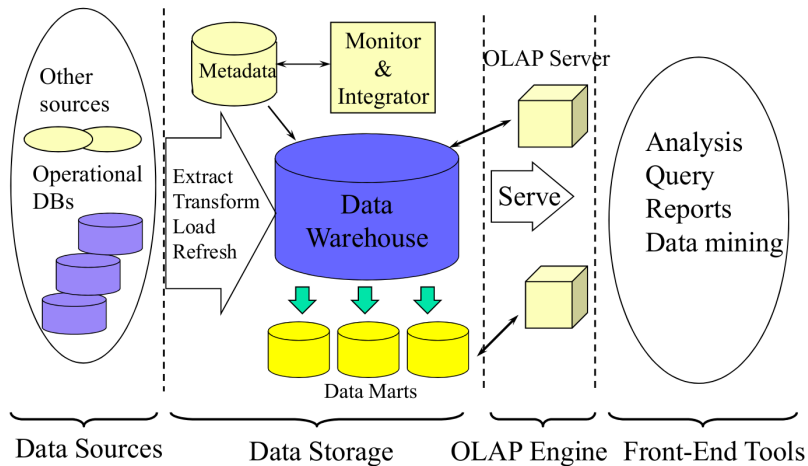
- ▶ Use pretrained deep neural networks
 - ▶ Conventionally, apply supervised training for individual structured fields, then use the network output as representation. This is actually one of the stacking approaches.
 - ▶ Recently, use self-supervised pretraining
 - ▶ BERT or GPT for text
 - ▶ ViT/DINO/iBOT for images
 - ▶ [HuggingFace](#) has many pretrained models for X2vec
- ▶ Pros and cons
 - + can extract comprehensive information
 - + from end to end
 - expensive in training time, tuning effort and computing resource
 - hard to explain so far
- ▶ NTNU IDI provides the following courses
 - ▶ IT3030 Deep Learning
 - ▶ TDT4310 Intelligent Text Analytics and Language Understanding
 - ▶ TDT4265 Computer Vision and Deep Learning

(Additional reading) Links

- ▶ The data samples can link to each other, e.g.
 - ▶ Web pages have hyperlinks
 - ▶ User have friend circles
 - ▶ Articles have citations
- ▶ Represent the network nodes (data samples) as vectors
 - ▶ A : network adjacency matrix; $D_{ii} = \sum_j A_{ij}$
 - ▶ Laplacian Eigenmap: $[V, \Lambda] = \text{eig} \left(I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \right)$
 - ▶ `sklearn.manifold.SpectralEmbedding`
 - ▶ Large-scale Information Network Embedding (LINE)
 - ▶ <https://github.com/tangjianpku/LINE>
 - ▶ Use Multi-dimensional Scaling (MDS) if the links are distances
 - ▶ `sklearn.manifold.MDS`
 - ▶ Recent Graphical Neural networks, e.g.,
 - ▶ Graph Convolutional Networks (GCN) or GraphSAGE

Manipulate tabular data

Data Warehouse



Pandas

- ▶ Pandas is the Python tool for data warehousing
- ▶ The name Pandas comes from the word Panel Data — an Econometrics from Multidimensional data.

Multidimensional data

- ▶ Data warehouse uses multidimensional data
- ▶ $v = f(x_1, x_2, \dots, x_d)$
 - ▶ $d = 1$: vector
 - ▶ $d = 2$: matrix (if all numeric) or tabular
 - ▶ $d = 3$: tensor (if all numeric)
- ▶ Most common format for *unstructured data*
 - ▶ Good mathematical tools
 - ▶ Computer friendly
- ▶ Pandas supports $d \leq 3$
- ▶ Tabular data is widely used in ML

Pandas data structure

- ▶ **Series** $\approx$ vector or 1D array

10	23	56	17	52	61	73	90	26	72
----	----	----	----	----	----	----	----	----	----

- ▶ **DataFrame** $\approx$ matrix, table, or 2D array, container of Series

Name	Age	Gender	Rating
Steve	32	Male	3.45
Lia	28	Female	4.6
Vin	45	Male	3.9
Katie	38	Female	2.78

- ▶ **Panel** $\approx$ cube or 3D array, container of DataFrame

Data interface

Extraction, Transformation, and Loading (ETL)

- ▶ Data extraction
 - ▶ get data from multiple, heterogeneous, and external sources
- ▶ Data cleaning
 - ▶ detect errors in the data and rectify them when possible
- ▶ Data transformation
 - ▶ convert data from legacy or host format to warehouse format
- ▶ Load
 - ▶ sort, summarize, consolidate, compute views, check integrity, and build indices and partitions
- ▶ Refresh
 - ▶ propagate the updates from the data sources to the warehouse

Data interface examples

Handy import from or export to various formats

- ▶ `pd.read_sql(sql, con)`: Read SQL query or database table into a DataFrame
Example

```
1 import pyodbc
2 conn = pyodbc.connect(r'Driver={Microsoft Access Driver (*.mdb, *.accdb)};'+
3                       r'DBQ=C:\Users\Zhirong\Desktop\sales.accdb;')
4 df = pd.read_sql('SELECT * FROM sales', self.conn)
```

- ▶ `pd.read_csv(filepath_or_buffer)`: Read a comma-separated values (csv) file into DataFrame.
 - ▶ accepts URL, including http, ftp, s3, and file
 - ▶ e.g. `pd.read_csv('data.csv')`
- ▶ `df.to_numpy()`: convert DataFrame to Numpy ndarray

A simple Pandas example

Pandas further reading

- ▶ 10 minutes to Pandas

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

- ▶ Tutorials point

https://www.tutorialspoint.com/python_pandas/

- ▶ Cheat sheet

https://github.com/pandas-dev/pandas/blob/master/doc/cheatsheet/Pandas_Cheat_Sheet.pdf

- ▶ More tutorials

https://pandas.pydata.org/pandas-docs/stable/getting_started/tutorials.html